



## Кейс 1. Генерация CRUD для системы заявок

---

В первом кейсе у нас есть достаточно типовая задача: нужно создать CRUD-модуль для сущности заявки. Здесь ИИ применим достаточно хорошо, потому что сама структура задачи стандартная. Есть сущность, есть набор полей, есть понятные операции создания, чтения, изменения и удаления, есть конкретный стек технологий и заранее заданные ограничения.

Если я передаю ИИ описание этой задачи, он вполне может подготовить первоначальный вариант реализации. Причём основная ценность здесь будет не в том, что ИИ придумает какую-то новую архитектуру, а в том, что он быстро соберёт типовую структуру, которую разработчик в противном случае писал бы вручную. Это как раз тот случай, где можно экономить время на шаблонной работе.

Я бы оценивал применимость ИИ здесь как высокую именно для подготовки первого рабочего варианта. CRUD-операции повторяются во множестве проектов, поэтому модель обычно хорошо распознаёт этот паттерн. Она может правильно связать маршруты, модели данных, обработку запросов и работу с базой данных, если ей передан достаточный контекст.

Но здесь сразу появляется важный риск. Типовая структура ещё не означает соответствие конкретному проекту. Например, в нашем кейсе запрещено изменять некоторые поля через запрос обновления. Если ИИ не учтёт это ограничение, он может сформировать слишком универсальную схему обновления и фактически разрешить изменение тех данных, которые должны оставаться неизменными.

То же самое касается статусов. В требованиях указаны конкретные допустимые значения. Если модель просто использует обычную строку без проверки, технически код может работать, но в базу данных начнут попадать некорректные состояния. Это уже будет не ошибка синтаксиса, а ошибка бизнес-логики.

Есть и риск, связанный с тем, что ИИ может сделать предположение о структуре проекта. Например, он может предложить расположение файлов, которое отличается от уже принятого в команде. Он может использовать другой способ организации зависимостей, другую схему обработки ошибок или другой подход к работе с сессиями базы данных.

Поэтому результат генерации нельзя воспринимать как готовый модуль, который достаточно просто вставить в проект. Здесь ИИ хорошо работает как ускоритель первоначальной реализации, но окончательная проверка всё равно должна выполняться разработчиком.

Если мы связываем этот кейс с метриками, то здесь полезно смотреть, насколько быстро разработчик получил первый пригодный вариант и какую часть результата действительно удалось использовать. Если ИИ за несколько секунд сгенерировал большую часть CRUD, но затем половину пришлось переписывать, экономия времени может оказаться значительно меньше ожидаемой.

Для этого кейса вывод будет такой: ИИ применим хорошо, потому что задача стандартная и формализованная. Основной риск связан не с неспособностью модели написать CRUD, а с тем, что типовая реализация может не учитывать конкретные правила проекта и бизнес-ограничения.

---

## Кейс 2. Миграция части проекта на новый стек

---

Во втором кейсе задача уже сложнее. Здесь мы не создаём функциональность с нуля, а переносим существующее поведение из одного технологического стека в другой.

Это важное отличие. При генерации нового CRUD у нас есть требования, на основании которых можно создать новый модуль. При миграции нам необходимо сохранить уже существующее поведение системы.

ИИ здесь тоже может быть полезен, но уровень риска становится выше.

Модель может достаточно быстро преобразовать структуру маршрута Flask в формат FastAPI. Она понимает оба фреймворка, поэтому техническое преобразование обычно не представляет большой сложности. Также она может адаптировать обработку ошибок, подключить схему ответа и перенести функцию в структуру нового приложения.

Но основная сложность миграции заключается не в замене одного синтаксиса другим.

В кейсе у нас явно указано, что URL должен остаться прежним, существующий сервис должен продолжить использоваться, а формат ответа нельзя произвольно менять, потому что его уже используют мобильное приложение и административная панель.

Если ИИ проигнорирует хотя бы одно из этих условий, мы получим технически корректный новый маршрут, который нарушит совместимость с существующей системой.

Например, модель может решить, что новый фреймворк позволяет сделать структуру ответа более аккуратной, и изменит формат возвращаемых данных. С точки зрения отдельного фрагмента кода это может выглядеть как улучшение. С точки зрения всей системы это может привести к тому, что клиентские приложения перестанут правильно обрабатывать ответ.

Именно поэтому при миграции я бы никогда не оценивал результат только по критерию «новый код запускается». Нужно проверять, сохраняется ли внешний контракт.

ИИ особенно полезен здесь как инструмент предварительного преобразования. Он может снять с разработчика часть рутинной работы, но задача проверки совместимости остаётся критически важной.

Есть ещё один риск: модель может начать переписывать больше, чем требуется. В нашем кейсе прямо сказано, что существующий сервис пока сохраняется. Но ИИ может решить, что правильнее сразу переделать и сервисный слой. Формально это может выглядеть логично, однако это уже выход за границы поставленной задачи.

Такое поведение особенно опасно в реальной разработке, потому что объём изменений напрямую влияет на вероятность появления дефектов. Чем больше кода меняется во время миграции, тем сложнее проверить, что поведение осталось прежним.

Поэтому здесь важный принцип заключается в том, что ИИ должен помогать переносить функциональность в заданных границах, а не самостоятельно расширять масштаб миграции.

---

Применимость ИИ в этом кейсе я бы оценивал как хорошую, но уже не настолько прямолинейную, как в CRUD. Чем сильнее задача зависит от сохранения существующего поведения и связей между компонентами, тем больше требуется человеческой проверки.

Основные риски здесь связаны с потерей обратной совместимости, изменением контракта API, выходом за границы задачи и скрытым изменением поведения системы.

## Кейс 3. Автодополнение при разработке функции

---

Третий кейс отличается от предыдущих тем, что здесь ИИ работает не по отдельному большому запросу, а постоянно присутствует в процессе написания кода.

Разработчик начинает функцию, а система пытается предугадать продолжение. В простом случае это очень удобно.

У нас есть цикл по товарам, есть цена, количество и итоговая сумма. Логика достаточно очевидная, поэтому система автодополнения с высокой вероятностью предложит корректное продолжение.

Здесь ИИ может реально ускорить работу, потому что разработчику не нужно полностью набирать типовой код вручную. Особенно хорошо это работает в повторяющихся конструкциях, где из окружающего контекста легко понять, что должно происходить дальше.

Но у автодополнения есть интересная особенность: его основной риск связан не только с качеством предложения, но и с поведением самого разработчика.

Когда система постоянно предлагает готовое продолжение, очень легко начать принимать подсказки автоматически. Разработчик видит, что код выглядит правдоподобно, нажимает принятие и продолжает работу.

Именно здесь может появиться проблема. Визуально правдоподобный код не обязательно соответствует требованиям.

В нашем случае задача достаточно простая, поэтому риск невелик. Но даже здесь система может предложить дополнительную обработку, которая сейчас не требуется, например начать учитывать скидки, хотя это явно вынесено за рамки текущей версии.

Также она может предположить, что поля всегда существуют, или предложить обработку данных, которая не соответствует реальному формату проекта.

Главное, что я хочу показать на этом кейсе: автодополнение не является обычной функцией текстового редактора. Это генеративная система. Она не знает, что разработчик действительно хотел написать. Она прогнозирует наиболее вероятное продолжение на основании контекста.

Поэтому каждую подсказку нужно воспринимать как предложение, а не как продолжение, которое гарантированно правильно.

Здесь особенно хорошо работает метрика доли принятых подсказок. Мы можем посмотреть, какую часть рекомендаций разработчик действительно использует.

---

Но сама по себе высокая доля принятия ещё ничего не доказывает. Если разработчик принимает почти все подсказки, это может означать как высокое качество системы, так и недостаточно внимательную проверку.

Более полезно оценивать, сколько принятых предложений потом приходится исправлять.

В этом кейсе ИИ наиболее применим для ускорения набора кода и завершения очевидных конструкций. Чем типичнее функция, тем выше вероятность полезного автодополнения.

Основной риск заключается в автоматическом доверии. Если разработчик перестаёт осмысленно читать предлагаемый код, автодополнение из инструмента ускорения превращается в источник незаметных ошибок.

## Кейс 4. Авторевью изменений перед слиянием

---

Четвёртый кейс уже связан не с написанием кода, а с его проверкой.

У нас есть Pull Request, в котором изменено несколько файлов и сотни строк. Человек-ревьюер должен изучить эти изменения и найти потенциальные проблемы.

ИИ здесь может использоваться как дополнительный уровень предварительной проверки.

Его сильная сторона состоит в том, что он может быстро просмотреть большой объём изменений и обратить внимание на подозрительные места.

Например, в нашем кейсе есть функция удаления пользователя. Если операция всегда сообщает об успешном удалении, независимо от того, существовал пользователь или нет, это может противоречить правилам проекта.

ИИ способен заметить такую ситуацию и сформировать комментарий.

Это полезно, потому что человек при просмотре нескольких сотен строк может случайно пропустить подобный участок.

Но здесь возникает ключевая проблема авторевью — ложные срабатывания.

ИИ может увидеть необычную конструкцию и решить, что она ошибочная, хотя на самом деле она сделана намеренно. Если таких замечаний будет слишком много, разработчики начнут их игнорировать.

Представьте, что на каждый Pull Request система оставляет двадцать комментариев, из которых действительно полезны два. Через некоторое время команда перестанет внимательно читать автоматические замечания.

Поэтому качество авторевью определяется не количеством найденных «проблем», а точностью и полезностью замечаний.

Есть ещё одна важная граница. Авторевью обычно видит только часть контекста.

---

Например, модель может анализировать изменённый метод, но не знать, что его необычное поведение требуется старым клиентским приложением. Или она может предложить изменить функцию, не учитывая договорённости, которые существуют в других частях проекта.

Чем меньше контекста передано ИИ, тем выше вероятность поверхностных или ошибочных замечаний.

Поэтому авторевью лучше всего использовать как дополнительный фильтр перед проверкой человеком.

Оно может найти очевидные дефекты, несогласованности, подозрительные условия и потенциальные нарушения правил проекта. Но окончательное решение о том, является ли замечание реальной проблемой, должно приниматься после проверки.

На этом кейсе особенно хорошо можно использовать метрику количества исправленных дефектов. Нас должно интересовать не то, сколько комментариев написал ИИ, а сколько реальных дефектов было подтверждено и устранено благодаря этим комментариям.

Если система оставила сто замечаний, но только три оказались полезными, высокая активность не означает высокую эффективность.

Если же она оставила десять замечаний, пять из которых помогли обнаружить реальные проблемы до слияния кода, это уже гораздо более значимый результат.

Для данного кейса применимость ИИ я бы оценивал как высокую в роли предварительного ревьюера, но не как замену человеческому ревью.

Основные риски здесь связаны с ложными срабатываниями, недостатком контекста, слишком большим количеством комментариев и ошибочным восприятием автоматического анализа как окончательной проверки.

Если сравнить все четыре кейса, становится видно, что применимость ИИ зависит прежде всего от характера задачи. Чем задача более типовая, локальная и хорошо формализованная, тем проще получить качественный результат. Чем сильнее задача зависит от существующей архитектуры, скрытых договорённостей, бизнес-логики и большого контекста проекта, тем осторожнее нужно относиться к результатам ИИ.

---